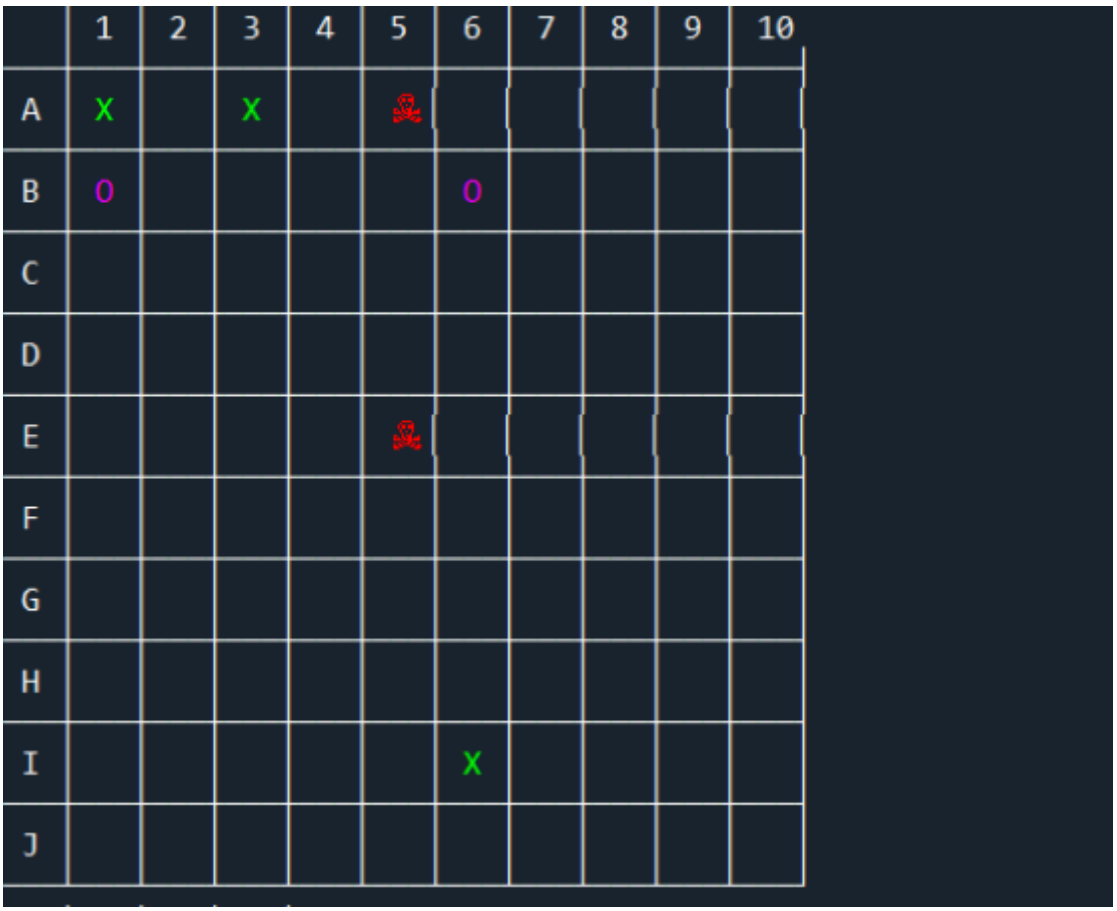


XO Brainer

Rapport

Par Guillaume Siossian et Ethan Diai

Introduction :

Nous avons choisi de réaliser le jeu « XO Brainer » en codage python sur le logiciel Spyder. L'objectif du projet était de développer ce jeu à l'aide des différentes séances de TP réalisées précédemment.

Voici les règles du jeu:

Dans le jeu XO Brainer vous devez réussir à créer une ligne de 5 éléments de même forme (X ou O) ou de même couleur (vert ou rose). Vous devez penser à utiliser, non seulement vos propres pièces, mais aussi celles de votre adversaire pour obtenir une ligne de 5. Lorsque cela est fait, vous marquez un certain nombre de points en fonction du placement de la ligne sur le plateau de jeu. La partie est gagnée quand vous avez marqué suffisamment de points ou lorsque vous gagnez par KO. La règle du KO est simple, il suffit d'aligner 5 pions sur une des deux cases KO qui se situent sur le plateau de jeu.

Structure du programme

Notre programme est structuré autour d'une fonction, la fonction bouclejeu. Celle-ci va coordonner toutes les fonctions de notre programme pour le faire fonctionner. Ainsi nos fonctions sont regroupées sous formes de parties :

Les fonctions qui génèrent les grilles de placement, de score : *Generer_Grille(), Generer_Grille2(), pointgrille()*

Les fonctions qui affichent les grilles de score, de placement et la grille avec les motifs restants : *Afficher_Grille(), Afficher_Grille2(), Afficher_Grille3().*

Les fonctions de saisie : *Saisie_Coords(),Coords2Nums()*. Nous les avons reprises puis modifiées du script de la bataille navale.

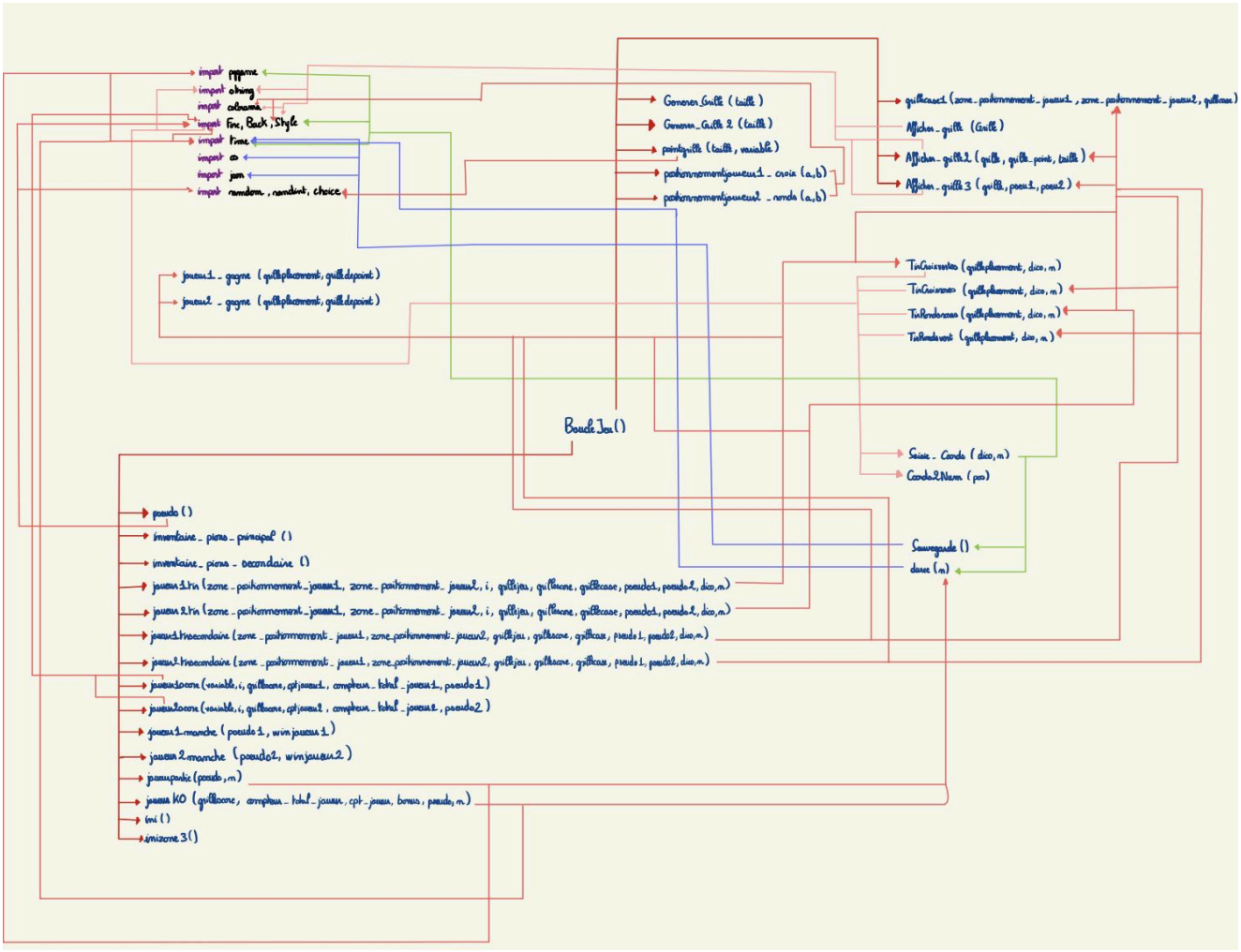
Les fonctions relatifs à chaque motifs utilisées par les fonctions d'affichage : *TirCroixvertes,TirCroixrose, TirRondsverts, TirRondsroses*

Les fonctions qui déterminent les conditions pour gagner : *joueur1_gagne() et joueur2_gagne().*

Les fonctions pour réinitialiser la grille de placement et la grille de motifs : *positionnementjoueur1_croix(), positionnementjoueur2_croix()*

La fonction de sauvegarde, la fonction qui donne la durée de la partie.

La fonction *bouclejeu* qui appelle les fonctions importantes pour faire fonctionner le jeu.



Variables importantes du projet

Dans la boucle jeu, nous utilisons une variable qui permet d'éviter au programme de rentrer inutilement dans des fonctions. Cette variable prend la valeur 1 lorsque le joueur 1 ou le joueur 2 gagne la manche. Admettons que le joueur 1 gagne la manche lorsque i = 2, variable prendra alors la valeur 1 et dans la suite du programme, comme variable = 1, il ne va pas dire au joueur 2 de sélectionner une case ou il devra mettre son pion. Vers la fin de la boucle, comme la variable vaut 1, le programme va réinitialiser tout ce qui est nécessaire pour refaire une manche.

```
    while zone_positionnement_joueur2 != [0,0,0,0]:
        if zone_positionnement_joueur2 != [0,0,0,0]:
            dico={"nom1":pseudo1,"nom2":pseudo2,"score1":compteur_total_joueur1,"score2":compteur_total_joueur2,"grille":grillescore,"zonej1":zone_positionnement_jou
            i = i+1
            #Afficher_Grille2(grillejeu,grillescore,10)
            if i in [0,1,2]:
                if variable != 1:
                    conditions_victoirs_joueur1, cptjoueur1, conditions_victoirs_joueur2, cptjoueur2 = joueur1tir(zone_positionnement_joueur1,zone_positionnement_jou


                if conditions_victoirs_joueur1 == 1:
                    compteur_total_joueur1, variable,i = joueur1score(variable, i, grillescore, cptjoueur1,compteur_total_joueur1,pseudo1)
                    if compteur_total_joueur1 >= 20:
                        winjoueur1 = joueur1manche(pseudo1,winjoueur1)
                        if winjoueur1 == 3:
                            joueurpartie(pseudo1, n)
                            return 1,pseudo1
```

```
def joueur1score(variable,i,grillescore,cptjoueur1,compteur_total_joueur1,pseudo1):
    variable = 1
    i = -2
    print(grillescore)
    compteur_total_joueur1 = compteur_total_joueur1 + cptjoueur1
    print(Back.RED+pseudo1 + " gagne la manche"+Style.RESET_ALL)
    print("le score de " + pseudo1 + " est " + str(compteur_total_joueur1))
    return compteur_total_joueur1, variable,i
```

La variable sera ensuite réinitialisée à 0.

Les variables du dictionnaire:
Notre partie est sauvegardé dans un dictionnaire, pour cela nous avons dû attribuer de nouvelles variables aux variables déjà existante:

dico={"nom1":pseudo1,"nom2":pseudo2,"score1":compteur_total_joueur1,"score2":compteur_total_joueur2,"grille":grillescore,"zonej1":zone_positionnement_joueur1,"zonej2":zone_positionnement_joueur2,"cpt1":cptjoueur1,"cpt2":cptjoueur2,"win1":winjoueur1,"win2":winjoueur2,"v":variable,"i1":i,"grillej":grillejeu,"grillec":grillecase,"icv":inventaire_croix_vertes,"icr":inventaire_croix_roses,"icv":inventaire_ronds_roses,"irv":inventaire_ronds_verts,"a":a,"b":b}

Il stocke ainsi le nom des joueurs, leurs scores, leurs emplacements sur la grille, le nombre de pions restants et de tours restants, la grille de score et la durée de la partie. Pour sauvegarder la partie, on importe le module json qui permet d'importer et d'exporter des informations dans un fichier texte.
La variable n:
Le module time, "time.time() permet d'avoir le nombre de secondes depuis l'époque (1er janvier 1970). Au début de la partie on stocke cette information dans la variable n qui sera utilisée à la fin de la partie par la fonction duree() pour comptabiliser la durée de la partie.

Fonctions du code

Beaucoup de fonctions sont nécessaires pour simplifier la programmation. On appelle ces fonctions à des moments stratégiques du code. Dans cette partie, on vous présentera les fonctions les plus importantes du programme pour vous donnez un aspect plus global de ce dernier.

Le code tourne autour de la fonction BoucleJeu() qui est la fonction centrale du programme. Voici le début de cette dernière :

```
def BoucleJeu():
    bonus=1000
    print(">>>>COMMANDES DU JEU<<<<<"+2*"\n"+"Arrêter la musique: taper "+Back.RED+"stop"+Style.RESET_ALL+"\n"+"Sauvegarder la partie: taper "+Back.GREEN+"pause"+Style.RESET_ALL)
    print(3*"\n"+"Voulez-vous poursuivre la partie précédente? (oui ou non)")
    rep=input()
    if rep=="oui":
        tf = open("myDictionary.json", "r")
        nvdico = json.load(tf)
        print(nvdico)
        a,b=nvdico["a"],nvdico["b"]
        print(Back.MAGENTA+"La partie reprends à ",a,"minute(s)"," et ",b," seconde(s)"+Style.RESET_ALL)
        pseudo1,pseudo2=nvdico["nom1"],nvdico["nom2"]
        print(Back.RED+"JOUEURS: ",pseudo1,"et ",pseudo2+Style.RESET_ALL)
        cptjoueur1,cptjoueur2,winjoueur1,winjoueur2,variable,zone_positionnement_joueur1,zone_positionnement_joueur2,compteur_total_joueur1,compteur_total_joueur2,grillescore,i,gri
        inventaire_croix_vertes,inventaire_croix_roses,inventaire_ronds_verts,inventaire_ronds_roses=nvdico["icv"],nvdico["icr"],nvdico["irv"],nvdico["irr"]
        n=int(time.time())
        grillecase1(zone_positionnement_joueur1,zone_positionnement_joueur2,grillecase)
        Afficher_Grille3(grillecase,pseudo1,pseudo2)
        Afficher_Grille2(grillejeu,grillescore,10)
    else:
        a,b=0,0
        n=int(time.time())
        pseudo1,pseudo2 = pseudo()
        cptjoueur1,cptjoueur2,winjoueur1,winjoueur2,variable,compteur_total_joueur1,compteur_total_joueur2,i = 0,0,0,0,0,0,0,-1
        grillecase = Generer_Grille2(4)
        grillejeu,grillescore,inventaire_croix_vertes,inventaire_croix_roses,inventaire_ronds_roses,inventaire_ronds_verts,zone_positionnement_joueur1,zone_positionnement_joueur2 =
        grillecase1(zone_positionnement_joueur1,zone_positionnement_joueur2,grillecase)
        print(zone_positionnement_joueur1),print(zone_positionnement_joueur2)
        Afficher_Grille3(grillecase,pseudo1,pseudo2)
        Afficher_Grille2(grillejeu,grillescore,10)
```

La fonction sauvegarde() permet de sauvegarder la partie. Elle utilise un système de dictionnaire pour stocker toutes les données.
Le dictionnaire stocke toutes les variables importantes utilisées dans la boucle jeu:

```
def sauvegarde(dico):
    tf = open("myDictionary.json", "w")
    json.dump(dico,tf)
    tf.close()
os.system("cls")
time.sleep(1)
print("chargement")
time.sleep(1)
```

Il stocke ainsi le nom des joueurs, leurs scores, leurs emplacements sur la grille, le nombre de pions restants et de tours restants, la grille de score et la durée de la partie. Pour sauvegarder la partie, on importe le module json qui permet d'importer et d'exporter des informations dans un fichier texte.

La fonction duree() permet d'obtenir le temps d'une partie :

```
def duree(n):
    w=int(time.time())
    y=w-int(n)
   # print(y)
    minutes = time.localtime(y)[4]
    secondes = time.localtime(y)[5]
    return minutes,secondes
```

Ici, la fonction fonctionne avec le module time, lorsqu'on appelle la fonction duree(n), celle-ci fait la différence entre "time.time() " actuel et n qui est le "time.time() " de debut de partie. On obtient ainsi le nombre de secondes écoulé et il ne reste plus qu'à utiliser time.localtime() pour convertir cette durée en minute et seconde.

Les fonctions positionnementjoueur1_croix(...) et positionnementjoueur2_ronds(...) permettent de définir correctement la zone de positionnement des joueurs 1 et 2, voici l'une de ces 2 fonctions :

```
def positionnementjoueur1_croix(a,b):#La grille de positionnement du joueur 1
    grille_positionnement_joueur1 = [0,0,0,0]
    inventaire_croix_vertes = a
    inventaire_croix_roses = b
    for i in range(0,4):
        grille_positionnement_joueur1[i] = 1  #Les trois premiere case de la grille sont des croix vertes
        inventaire_croix_vertes = inventaire_croix_vertes - 1
        if i == 3 :
            grille_positionnement_joueur1[3] = 2  #la quatrieme case de la grille possède une croix rose.
            inventaire_croix_roses = inventaire_croix_roses - 1
        if inventaire_croix_vertes == 0 and inventaire_croix_roses == 0:
            break  #peut etre mettre une fonction qui dit que la manche ce termine
    print("Il reste " + Fore.GREEN +str(inventaire_croix_vertes) + " croix verte(s) au joueur 1"+Style.RESET_ALL)
    print("Il reste " +Fore.MAGENTA+ str(inventaire_croix_roses) + " croix rose(s) au joueur 1"+Style.RESET_ALL)
    return grille_positionnement_joueur1
```

Les fonctions Generer_Grille s'occupent de générer les grilles nécessaires au bon fonctionnement du jeu. On aura donc la grille ou les joueurs jouent, la grille de positionnement des joueurs et la grille qui attribuent des valeurs aux cases. Voici par exemple la grille qui attribuent des valeurs aux cases :

```python
def pointgrille(taille, variable):
    grille_point = Generer_Grille(taille)
    for i in range(taille):
        for j in range(taille):
            grille_point[i][j] = randint(0, 4)
    while variable == True:
        a,b = randint(0,9),randint(0,9)
        c,d = randint(0,9),randint(0,9)
        grille_point[a][b],grille_point[c][d] = 5,5
        if a != c or b != d:
            variable = False
    print(a,b)
    return grille_point  #La grille retournée est une grille
```

Les fonctions de type Afficher_Grille s'occupent d'afficher les grilles aux utilisateurs. Elles apportent certains aspects graphiques (couleurs, motifs…) pour rendre le jeu plus agréable en plus de rendre le jeu jouable. La luminosité des pions est augmenté grâce au module colorama et à "Style.BRIGHT". Voici par exemple la fonction qui permet aux utilisateur de voir le plateau du jeu :

Point de vue de chacun :

Guillaume :

Ce projet a énormément amélioré mon niveau sur python, il a même je dirais déclenché une appétence pour le codage. J'ai développé des compétences nouvelles. Par exemple, j'ai pu apprendre à utiliser de nombreux modules qui m'étaient inconnus avant (pygame, colorama, time). Ce projet a été pour moi, plus un plaisir, qu'une corvée. Je suis assez content du travail réalisé avec mon binôme, nous avons tous les deux étés aussi investis. Notre programme a très vite fonctionné c'est pourquoi nous avons eu le temps d'ajouter quelques fonctionnalités comme le comptage de la durée d'une partie ou la musique.

```python
def Afficher_Grille2(grille, grille_point, taille):
    ligne = "   |" + "|".join(f" {i} " for i in range(1, taille + 1))
    print(ligne)
    for k in range(taille):
        ligne_inter = "———┼" * taille + "———|"
        print(ligne_inter)
        ligne = f" {chr(65+k)} |"
        for j in range(taille):
            if grille[k][j] == 1:
                ligne += Style.BRIGHT+"\033[32m X \033[0m|" +Style.RESET_ALL # croix vertes
            elif grille[k][j] == 2:
                ligne += Style.BRIGHT+"\033[35m X \033[0m|"+Style.RESET_ALL  # croix roses
            elif grille[k][j] == 3:
                ligne += Style.BRIGHT+"\033[35m O \033[0m|"+Style.RESET_ALL  # ronds roses
            elif grille[k][j] == 4:
                ligne += Style.BRIGHT+"\033[32m O \033[0m|"+Style.RESET_ALL  # ronds verts
            elif grille_point[k][j] == 5:
                ligne += Style.BRIGHT+"\033[31m \033[0m|"+Style.RESET_ALL

            else:
                ligne += "   |"  # cases vides
        print(ligne)
    ligne_inter = "———┴" * taille + "———|"
    print(ligne_inter)
    return grille
```

Le joueur à besoin pour placer ses pions de saisir des coordonnées. Ces coordonnées sous format alphanumérique sont ensuite convertie par le programme pour ensuite placer le pion. Voici cette fonction :

```
def Saisie_Coords(dico,n):
    print(Back.BLUE+"Saisir une lettre entre A et J et un chiffre entre 1 et 10"+Style.RESET_ALL)
    alph=["A","B","C","D","E","F","G","H","I","J"]
    coords_valides = [f"{l}{n}" for l in alph for n in range(1, 11)]
    while True:
        saisie = input().upper()
        if saisie in coords_valides:
            return saisie
        if saisie=="STOP":
           pygame.mixer.music.stop()
           print("plus de musique")
        if saisie=="PAUSE":
            u,i=duree(n)
            dico["a"],dico["b"]=dico["a"]+u,dico["b"]+i
            print("La partie est sauvegardée à ", dico["a"],"minute(s)"," et ",dico["b"]," seconde(s)")
            sauvegarde(dico)
            pygame.mixer.music.stop()
            print("Votre partie a été sauvegardée")
            time.sleep(5)
            break


        else:
            print(Back.RED+"Case non disponible, veuillez taper une case valide !"+Style.RESET_ALL)
```

C’est à cette fonction que le joueur peut rentrer des informations et interagir avec le programme. C’est donc dans cette fonction que nous laissons la possibilité aux joueurs de stopper la musique et de sauvegarder la partie. Si le joueur tape “pause”, nous enregistrons dans le dictionnaire la durée de la partie en cours avec l’appel à la fonction duree() puis nous appelons sauvegarde(). Pour que les joueurs puissent lire le message affiché avant l’arrêt du programme (break), nous laissons un délai de 5 secondes.

Pour attribuer à chaque placement un pion du joueur 1 ou du joueur 2, on utilise les fonctions de type Tir. Grâce à ces fonctions, le programme est capable de différencier le placement d’une croix verte ou rose et le placement d’un rond vert ou rose. Voici par exemple la fonction qui permet de placer une croix verte :

```
def TirCroixvertes(grilleplacement,dico,n):
    for i in range (1000):
        coordonnée_joueur1 = Saisie_Coords(dico,n)
        ligne,colonne=Coords2Nums(coordonnée_joueur1)
        position = grilleplacement[ligne][colonne]
        if position == 0:
            grilleplacement[ligne][colonne] = 1
            return grilleplacement
        else:
            if position in [1, 2, 3, 4]:
                print(Back.RED+"case déjà occupée, rééssayer"+Style.RESET_ALL)
```

Le code tout seul n’est évidemment pas capable de savoir les conditions nécessaires à la victoire d’un joueur. Il y a évidemment le type de pion, mais aussi la position relative des pions et la valeurs de la case associée (case KO, case de 4 points, etc). Il faut également que le programme compte le nombre de points que le joueur à gagné s' il à réussi à remplir les conditions pour gagner. Pour ce faire, nous avons créé les fonctions joueur1gagne et joueur2gagne qui accomplissent ces différentes tâches. Voici par exemple le début de la fonction associé au joueur 1 et qui vérifie si ce dernier à effectivement gagner :

```
def joueur1_gagne(grilleplacement,grilledepoint):
    compteur_de_point=0
    for i in range(0,9):
        for j in range(0,9):
            if (grilleplacement[i][j]==1 or grilleplacement[i][j]==1) and ( grilleplacement[i][j+1]==1 or grilleplacement[i][j+1]==2 )and( grilleplacement[i][j+2]==1
                for x in range(0,4) :
                    compteur_de_point = grilledepoint[i][j+x]+  compteur_de_point
                    if grilledepoint[i][j+x]==5 :
                        print("GAGNE PAR KO!")
                        return 10, compteur_de_point
                print("5 croix vertes sont alignées, gagné!")
                return 1, compteur_de_point
    for i in range(0,9):
        for j in range(0,9):
            if( grilleplacement[i][j]==1 or grilleplacement[i][j]==4) and ( grilleplacement[i][j+1]==1 or grilleplacement[i][j+1]==4 )and( grilleplacement[i][j+2]==1
                            compteur_de_point = grilledepoint[i][j]+ grilledepoint[i][j+1] + grilledepoint[i][j+2] + grilledepoint[i][j+3] + grilledepoint[i][j+4]
                            print("5 croix vertes sont alignées, gagné!")
                            return 1, compteur_de_point
```

```
or grilleplacement[i][j+2]==2 )and ( grilleplacement[i][j+3]==1 or grilleplacement[i][j+3]==2)and (grilleplacement[i][j+4]==1 or grilleplacement[i][j+4]==2):




or grilleplacement[i][j+2]==4)and ( grilleplacement[i][j+3]==1 or grilleplacement[i][j+3]==4 )and (grilleplacement[i][j+4]==1 or grilleplacement[i][j+4]==4):
```

Cette fonction a été plusieurs fois simplifiée pour obtenir cette version moins synthétique. Cette fonction retourne 1 pour une victoire simple et 10 pour une victoire par KO.

Récapitulatif des fonctionnalités

Notre jeu a toutes les fonctionnalités attendues et même un peu plus.

fonctionnalitées:

- Tirage au sort des joueurs
- Génération des grilles
- Cases KO placées aléatoirement sur la grille de placement
- Affichage de la grille de point, de la grille de placement, de la grille de pions
- Inventaire des pions à jouer et des pions restants
- Changement de grille de pions tout les quatre pions joués
- Victoire selon les conditions du jeu (si 5 croix alignées (verte ou rose), ou cinq ronds alignés)
- Victoire comptabilisées pour alignement en ligne en colonne et en diagonale
- Victoire comptabilisées par KO
- Les compteurs de point stockent le nombre de points gagnés en cas de victoire
- Couleurs utilisés pour rendre le jeu plus agréable
- A chaque victoire: affichage du score, nouvelle grille de point, de placement , de pions, et nouvelles cases KO
- Sauvegarde fonctionnelle de la partie
- Comptabilisation de la durée de la partie
- Lorsque le jeu est sauvegardée, la durée de la partie précédente est stockée pour être comptabilisée en cas de reprise
- Lancement d’une musique de fond qu’il est possible de stopper à tout moment du jeu

Le jeu s’arrête:

- en cas de victoire
- en cas de victoire par KO
- lorsque un joueur tape ”pause” et sauvegarde la partie